

CSE 331

AUTOMATA AND COMPUTABILITY

Summary

Summer 2026

Today's learning goals

- Summarize key concepts, ideas, themes from CSE331.
- Approach your mid exam studying with confidence.
- Identify areas to focus on while studying for the mid.

Vocabulary review

Sipser p. 14

- $\{a, b, c, d, e\}$ The set whose elements are a, b, c, d, e
- $\{a, b\}^*$ The set of finite strings over a, b
 - Includes empty string ϵ
 - Includes a, aa, aaa
 - Includes b, bb, bbb
 - Includes $ab, ababab, aaaaaaabb$
 - Does **not** include infinite sequences of a 's and b 's
 - Has **infinitely many** different elements
- $|ababab| = 6$ The length of the string $ababab$ is 6
- $|\{a, b, c, d, e\}| = 5$ The size of the set $\{a, b, c, d, e\}$ is 5
- A **language** (over alphabet Σ) is a set of strings (over Σ)

Deterministic finite automaton

Sipser p. 35 Def 1.5

The finite automaton M_1

We can describe M_1 formally by writing $M_1 = (Q, \Sigma, \delta, q_1, F)$, where

1. $Q = \{q_1, q_2, q_3\}$,
2. $\Sigma = \{0, 1\}$,
3. δ is described as

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 is the start state, and
5. $F = \{q_2\}$.

If A is the set of all strings that machine M accepts, we say that A is the *language of machine M* and write $L(M) = A$. We say that M *recognizes A* or that M *accepts A* . Because the term *accept* has different meanings when we refer to machines accepting strings and machines accepting languages, we prefer the term *recognize* for languages in order to avoid confusion.

A machine may accept several strings, but it always recognizes only one language. If the machine accepts no strings, it still recognizes one language—namely, the empty language $\emptyset$.

Deterministic finite automaton

Sipser p. 35 Def 1.5

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.



**No circles and
arrows, same
information!**

Building DFA



Typical questions

Define a DFA which recognizes the given language L .

or

Prove that the (given) language L is regular.

Building DFA



Remember

States are our only (computer) memory.

Design and pick states with specific roles / tasks in mind.

"Have not seen any of desired pattern yet"

"Trap state"

Justification?

To prove that the DFA we build, M , actually recognizes the language L

$$\text{WTS } L(M) = L$$

(1) Is every string accepted by M in L ?

(2) Is every string from L accepted by M ?

or contrapositive version: Is every string rejected by M not in L .

Regular languages

Sipser p. 35 Def 1.5

- For DFA M over the alphabet Σ
 - For each string w over Σ , M either accepts w or rejects w
 - The **language of M** is the set of strings M accepts (also called **language recognized by M**)
 - **$L(M)$** = $\{ w \mid w \text{ is a string over } \Sigma \text{ and } M \text{ accepts } w \}$

Definition: A language is **regular** iff there is some finite automaton that recognizes **exactly** it.

Given L , prove it is regular

Construction

Strategy 1: Construct DFA

Strategy 2: Construct NFA

Strategy 3: Construct regular expression

Proof of correctness

WTS 1 if w is in L then w is accepted by

WTS 2 if w is not in L then w is rejected by ...

Proving closure

Goal: "The class of _____ languages is closed under _____"

In other words Given a language in specific class, is the result of applying the operation _____ to this language still guaranteed to be in the class?

Proving closure

Given: What does it mean for L to be in class?

e.g. L a regular language, so given a DFA $M_L = (Q_L, \Sigma_L, \delta_L, q_L, F_L)$
with $L(M_L) = L$. Name each of the pieces!

WTS: The result of applying the operation to L is still in this class.

Construction: Build a machine that recognizes the result of applying the operation to L . Start with description in English!

e.g. Let $M = (Q, \Sigma, \delta, q_0, F)$ where $Q = \dots \Sigma = \dots \delta = \dots q_0 = \dots F = \dots$
 M could be DFA or NFA

Correctness: Prove $L(M) = \text{result of applying operation to } L$

WTS1 if w is in set then w is accepted by M

WTS2 if w is not in the set then w rejected by M .

Complementation

Theorem: If A is a regular language over $\{0,1\}$, then so is its complement $\bar{A}$

aka "the class of regular languages is closed under complementation"

Proof: Let A be a regular language.

Then there is a DFA $M=(Q,\Sigma,\delta,q_0,F)$ such that $L(M) = A$.

We want to build a DFA M' whose language is $\bar{A}$. Define

$M' =$

?

Claim of Correctness $L(M') = \bar{A}$

Proof of claim...

The regular operations

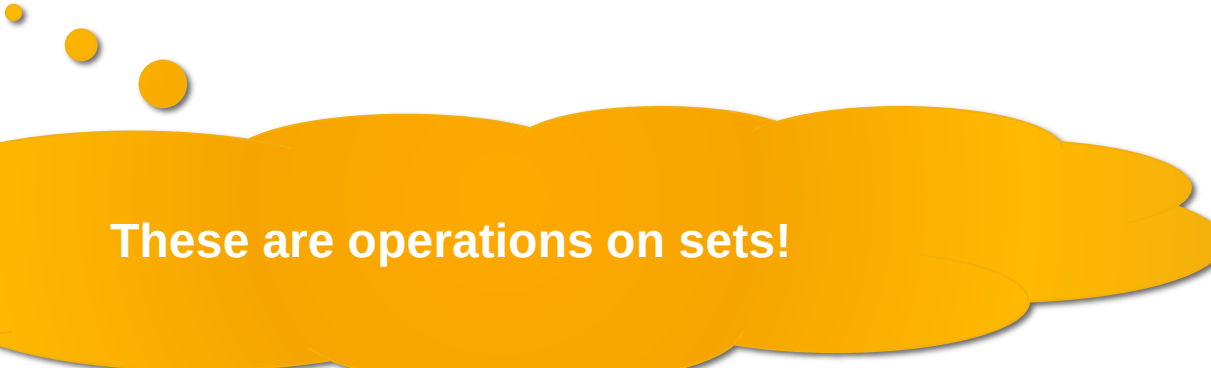
Sipser Def 1.23 p. 44

For A, B languages over same alphabet, define:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$

$$A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$$



These are operations on sets!

Union

Sipser Theorem 1.25 p. 45

Theorem: The class of regular languages over fixed alphabet Σ is closed under the union operation.

Proof: Let A_1, A_2 be any two regular languages over Σ .

WTS that $A_1 \cup A_2$ is regular.

Goal: build a DFA that recognizes $A_1 \cup A_2$.

Union

Sipser Theorem 1.25 p. 45

Theorem: The class of regular languages over fixed alphabet Σ is closed under the union operation.

Proof: Let A_1, A_2 be any two regular languages over Σ .

Given $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ such that $L(M_1) = A_1$ and

$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ such that $L(M_2) = A_2$ and

WTS that $A_1 \cup A_2$ is regular.

Define $M = (Q_1 \times Q_2, \Sigma, \delta, ?, ?)$

General proof structure/strategy

Theorem: For any L over Σ , if L is regular then [the result of some operation on L] is also regular.

Proof:

Given name variables for sets, DFAs assumed to exist.

WTS state goal and outline plan.

Construction using objects previously defined + new tools working towards goal. Give formal definition and explain.

Correctness prove that construction works.

Conclusion recap what you've proved.

Difference between NFA and DFA

- **DFA**: unique computation path for each input
- **NFA**: allow several (or zero) alternative computations on *same input*
 - $\delta(q,x)$ may specify **more than one** possible next states
 - ϵ transitions allow the machine to **transition between states spontaneously**, without consuming any input symbols
- Types of components of formal definition
 - **DFA** $\delta : Q \times \Sigma \rightarrow Q$
 - **NFA** $\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$

Formal definition of NFA

A **nondeterministic finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.

Similarities between DFA and NFA

- If L is a language recognized by a DFA, is there some NFA that recognizes it?
 - A. Yes
 - B. No
 - C. Depends on L
 - D. I don't know.

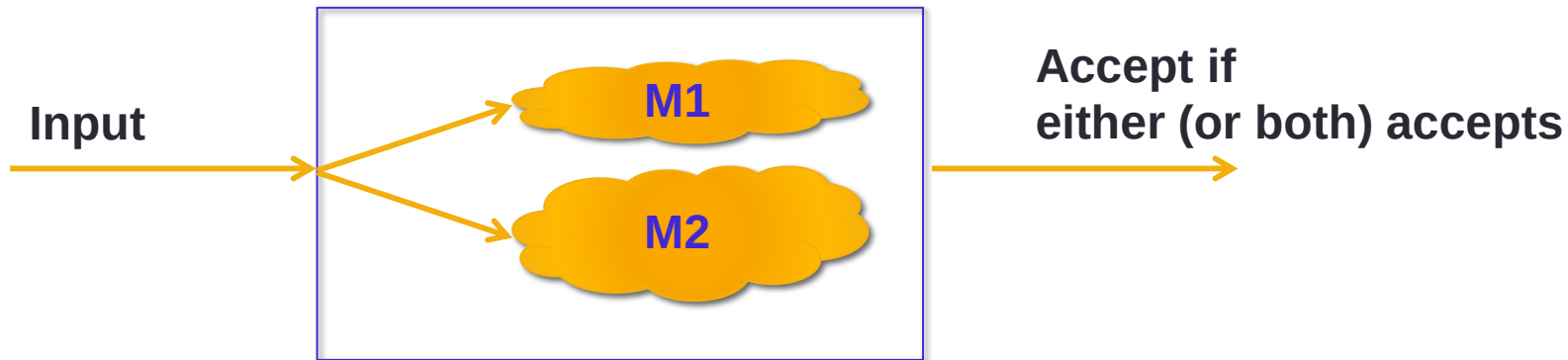
Ex: $L = \{ w \text{ in } \{0,1\}^* \mid w \text{ has odd \# of 1s OR starts with 0} \}$

NFA:

Regular expression:

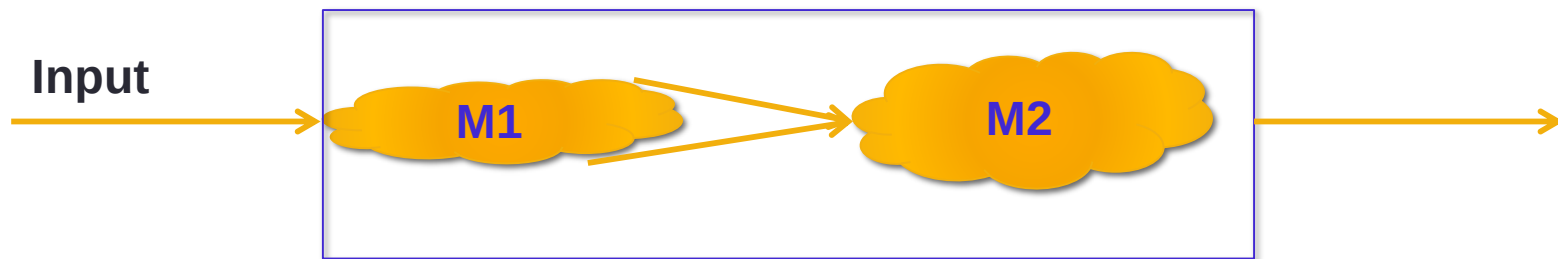
New – easier – construction for union

- "Guess" one of finite list of criteria to meet



Concatenation

- "Guess" some stage of input at which switch modes



Given DFAs $M1 = (Q1, \Sigma, \delta1, q1, F1)$, $M2 = (Q2, \Sigma, \delta2, q2, F2)$

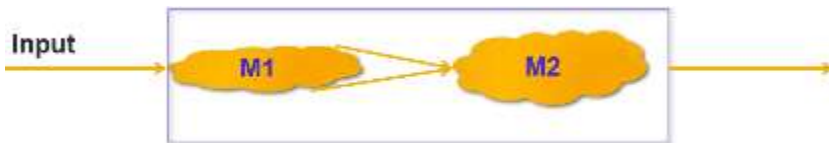
build NFA $N = (Q1 \cup Q2, \Sigma, \delta, q1, F2)$ with $\delta \dots$

Concatenation

PROOF

Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 , and
 $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 .

Construct $N = (Q, \Sigma, \delta, q_1, F_2)$ to recognize $A_1 \circ A_2$.



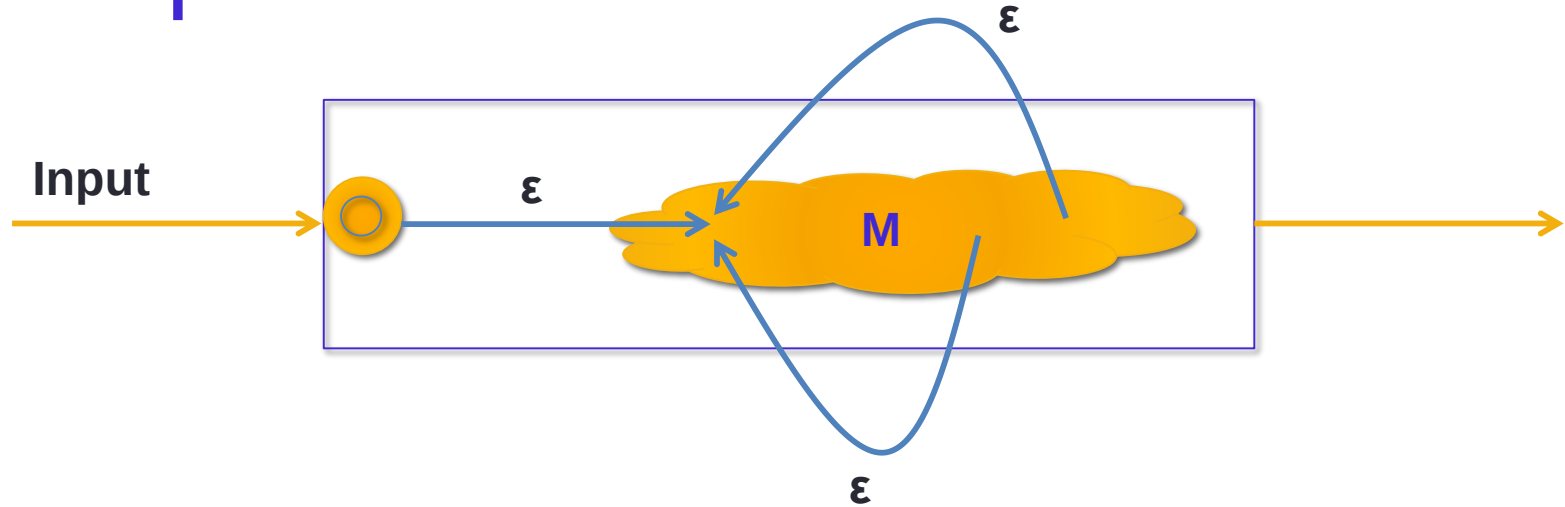
1. $Q = Q_1 \cup Q_2$.

The states of N are all the states of N_1 and N_2 .

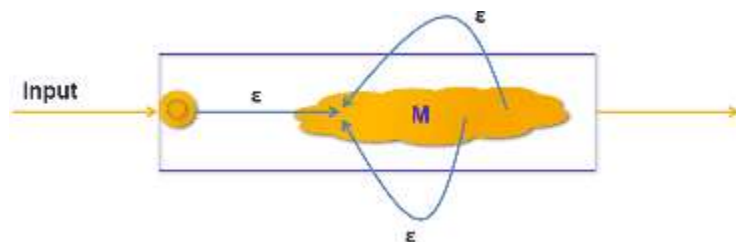
2. The state q_1 is the same as the start state of N_1 .
3. The accept states F_2 are the same as the accept states of N_2 .
4. Define δ so that for any $q \in Q$ and any $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ and } a = \epsilon \\ \delta_2(q, a) & q \in Q_2. \end{cases}$$

Star operation



Star operation



PROOF Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 .
Construct $N = (Q, \Sigma, \delta, q_0, F)$ to recognize A_1^* .

1. $Q = \{q_0\} \cup Q_1$.

The states of N are the states of N_1 plus a new start state.

2. The state q_0 is the new start state.

3. $F = \{q_0\} \cup F_1$.

The accept states are the old accept states plus the new start state.

4. Define δ so that for any $q \in Q$ and any $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \epsilon \\ \{q_1\} & q = q_0 \text{ and } a = \epsilon \\ \emptyset & q = q_0 \text{ and } a \neq \epsilon. \end{cases}$$

Simulating NFA with DFA

Not quite a closure proof, but ...

Proof:

Given name variables for sets, machines assumed to exist.

WTS state goal and outline plan.

Construction using objects previously defined + new tools working towards goal. Give formal definition and explain.

Correctness prove that construction works.

Conclusion recap what you've proved.

Subset construction

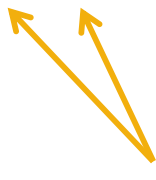
Given A , a language recognized by NFA $N = (Q, \Sigma, \delta, q_0, F)$

WTS there exists a DFA M with $L(M) = A$

Construction Define $M = (Q', \Sigma, \delta', q_0', F')$ with

- $Q' =$ the power set of $Q = \{ X \mid X \text{ is a subset of } Q \}$
- $q_0' = \delta^*(q_0, \epsilon)$
- $F' = \{ X \mid X \text{ is a subset of } Q \text{ and } X \cap F \text{ is nonempty} \}$
- $\delta' ((X, x)) = \{ q \text{ in } Q \mid q \text{ is in } \delta(r, x) \text{ for some } r \text{ in } X \text{ or accessible via spontaneous moves} \}$

Types?



DFAs vs Regular Expressions

- DFAs – **check** if a given word is in the language
- Regular Expressions – **generate** all works in the language

Inductive application of closure

Sipser 1.52 p. 64

R is a **regular expression** over Σ if

1. $R = a$, where $a \in \Sigma$
2. $R = \varepsilon$
3. $R = \emptyset$
4. $R = (R_1 \cup R_2)$, where R_1, R_2 are themselves regular expressions
5. $R = (R_1 \circ R_2)$, where R_1, R_2 are themselves regular expressions
6. (R_1^*) , where R_1 is a regular expression.

Watch out for
overloaded symbols!

Σ is shorthand for $(0 \cup 1)$ if $\Sigma = \{0, 1\}$, Parentheses may be omitted, R^+ means RR^* , R^k means R concatenated with itself k times

L(R)

Let L be the language over {a,b} described by the regular expression

$$((a \cup \emptyset) b^*)^*$$

Which of the following is **not** true about L?

- A. Some strings in L have equal numbers of a's and b's
- B. L contains the string aaaaaa
- C. a's never follow b's in any string in L
- D. L can also be represented by the regular expression $(ab^*)^*$
- E. More than one of the above.

"Regular = regular"

Sipser Theorem 1.54 p. 66

Theorem: A language is regular if and only if some regular expression describes it.

Lemma 1.55: If a language is described by a regular expression, then it is regular.

Lemma 1.60: If a language is regular, then it is described by some regular expression

regexp to NFA (to DFA)

Lemma 1.55, page 67

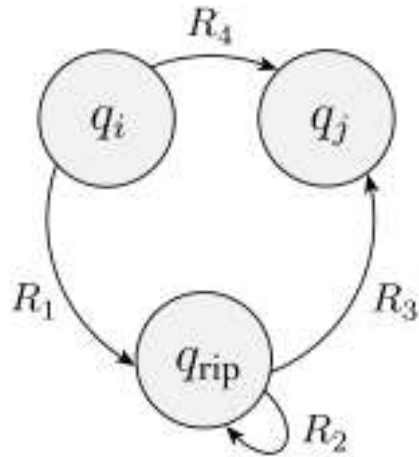
- Idea: basic regular expressions are easy to implement as DFA, for inductive step of definition, use closure under regular operations.
- Example: build NFA for the language described by $(00 \cup 11)^*$

DFA to GNFA example

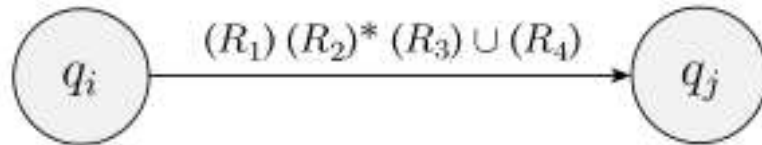
- Convert DFA to GNFA
- Add new states: **start state, accept state**
- Add edges from **new start state** to **original start state**
- Add edges from **original accept states** to **new accept state**
- Replace edge labels with basic regexps

GNFA simplification

- Simplify GNFA by removing one node at a time



before



after

Model of computation <i>Formal definition?</i> <i>Design?</i> <i>Describe language?</i>	Class of languages <i>Closure properties?</i> <i>Which languages not in class?</i>
Finite automata -- DFA -- NFA equiv to Regular expressions	Regular languages

Closure properties

	Regular Languages
Union	✓
Intersection	✓
Complement	✓
Star	✓
Concatenation	✓

Advice studying for the mid

- Don't stress!
- Go over lecture notes
- Go over homework sets and solutions
 - Try to solve all homework problems on your own first
 - Then, make sure you understand the solutions
- Read in the book if you need extra details

